WOD2ALPAsim: Extending AlpaSim into a Closed-Loop Harness for WOD-Style Drivers

Alba Maria Tellez Fernandez
amtellezfernandez@gmail.com

Abstract: Closed-loop driving evaluation is often blocked by a practical but consequential mismatch: datasets define one policy contract, while simulators execute another. The Waymo Open Dataset end-to-end driving task exposes camera context, ego history, route intent, and a fixed five-second trajectory target. AlpaSim executes native trajectory-model plugins inside a simulator runtime with camera streams, ego dynamics, route updates, controller execution, and deployment-specific service orchestration. This paper presents WOD2ALPAsim, a full-stack adapter that made WOD-style and minimal-shot driving policies run as AlpaSim external drivers. Although WOD is the motivating dataset contract, the artifact also establishes a reusable adapter pattern for future AlpaSim integrations: open only simulator-stable driver seams, reconstruct a compact policy signal outside dataset-specific protobufs, register policies through the existing model-plugin interface, and make deployment state executable through setup and launch scripts. The artifact is not only a wrapper around policy code: it patches the AlpaSim driver contract to pass route waypoints into model prediction, hardens the driver service against late simulator messages, makes model imports lazy so external plugins can load without heavyweight built-in backends, adds deploy/runtime overrides for local external-driver execution, registers three project policies through the `alpasim.models` plugin interface, reconstructs route and scene signals from AlpaSim prediction inputs, and provides reproducible setup, readiness, and launch scripts. This turns AlpaSim from a simulator that can run its built-in driver stack into a harness for WOD-style trajectory policies, learned token policies, and actor-aware planner variants under the same closed-loop runtime.

Keywords: autonomous driving, simulation, Waymo Open Dataset, closed-loop evaluation, AlpaSim, policy adapters

1 Introduction

The Waymo Open Dataset (WOD) end-to-end driving task is useful because it gives the community a concrete policy interface: observe camera and ego context, condition on route intent, and output a short-horizon ego trajectory [1]. That interface is not the same thing as a simulator driver. AlpaSim is designed around live closed-loop driver services and trajectory-model plugins [2]. A WOD-style policy can emit a trajectory, but AlpaSim expects a driver that can be discovered as a model plugin, accept simulator-native camera and egomotion messages, receive route updates, return headings and trajectory points at the configured rollout clock, and survive the ordering behavior of a distributed simulator runtime.

The work in WOD2ALPAsim is therefore not ”convert a log into a scene.” The central contribution is an execution bridge: we changed the AlpaSim-facing contract and the project policy stack until WOD-style policies could be launched as first-class AlpaSim external drivers. This required modifications on both sides of the boundary. On the AlpaSim side, route waypoints had to be preserved beyond

10th Conference on Robot Learning (CoRL 2026), Austin TX, USA.

Surface	Stock AlpaSim behavior	WOD2ALPAsim behavior
Route input	route used mainly for command	route waypoints preserved for prediction
Model imports	eager built-in backend imports	lazy imports for external plugin envs
Plugin boundary	built-in model stack	local `alpasim.models` policies
Session messages	unknown-session errors	late messages handled idempotently
Trajectory output	simulator frequency contract	WOD-style 5 s horizon resampled to runtime
Deployment	wizard-driven built-ins	matched external driver and wizard commands
Host setup	monolithic dependency assumptions	minimal editable install and overrides

Table 1: The adapter changes both the AlpaSim-side execution surface and the project-side policy surface.

command classification and passed through `PredictionInput`; model import had to tolerate local-driver environments without Alpamayo or VAM installed; late close, image, egomotion, and route messages had to be treated idempotently; and local external-driver deployment needed repeatable Docker, wizard, and host-environment setup. On the policy side, our minimal-shot, token-BC/DAgger, and direct actor-planner variants had to become native AlpaSim model plugins while still preserving their WOD-style route, timing, and trajectory assumptions.

The resulting artifact is a systems contribution suitable for a workshop because it exposes a practical path from dataset-style policy development to closed-loop simulator execution. More importantly, it sets a precedent for future AlpaSim adapters. The WOD case reveals which seams need to be opened for external policies in general: route geometry must survive into prediction, plugin discovery must not require unrelated built-in model dependencies, distributed-runtime messages must be handled robustly, and launch assumptions must be captured as executable state. The paper can therefore state which parts are patched upstream AlpaSim, which parts are project-owned adapters, and which runtime scripts make the result reproducible.

Contributions. We contribute:

1. a comparison between stock AlpaSim's trajectory-driver contract and the contract needed by WOD-style policies;
2. AlpaSim-side patches that expose route waypoints to model prediction, support dependency-light external plugins, harden asynchronous driver sessions, and adapt local deployment;
3. project-side AlpaSim model plugins for a minimal-shot reflex policy, a learned token BC/DAgger policy, and a selector-free actor-aware planner;
4. a signal reconstruction layer that turns AlpaSim prediction inputs into route, dynamics, visibility, and optional actor/hazard context; and
5. a reusable adapter pattern for future AlpaSim bridges, with a reproducible setup and launch path that installs the local plugin into an AlpaSim checkout, verifies registry discovery, writes matched driver/wizard commands, runs named scene presets, and checks artifact invariants through unit tests and readiness scripts.

2 Why a Thin Adapter Was Not Enough

Stock AlpaSim already has the right high-level architecture for this task: a runtime queries an ego-driver service, the driver returns a trajectory, and downstream controller and vehicle components execute it. The mismatch appeared in the details of the driver contract and deployment path. Table 1 summarizes the difference.

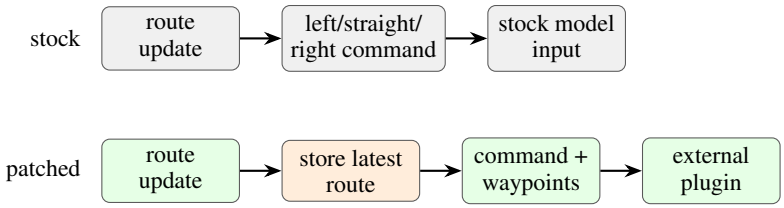

Figure 1: The key AlpaSim driver-contract patch preserves route waypoints for prediction instead of reducing route input to a command-only signal.

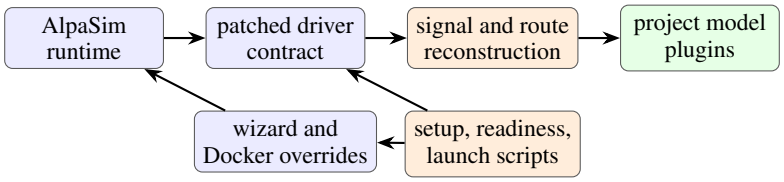

Figure 2: WOD2ALPAsim is a full-stack integration: patched AlpaSim driver and deployment surfaces feed project-owned model plugins through a reconstructed policy signal.

The most important issue was route information. In the original driver API, `PredictionInput` contained camera images, command, speed, acceleration, and ego pose history. That is enough for a command-conditioned model, but not for policies that need the local route geometry. WOD2ALPAsim stores the latest route submitted by the runtime, copies its waypoints into `PredictionInput`, and lets model plugins build a local route centerline from simulator waypoints. This is a small patch in code size and a large change in capability: policies can now reason over the actual AlpaSim route instead of only a left/straight/right symbol.

The second issue was plugin loading. A local external-driver environment may not contain every heavy dependency needed by AlpaSim's built-in learned models. Eagerly importing all built-ins blocks unrelated external plugins. WOD2ALPAsim replaces that import path with lazy loading: the base abstractions remain importable, the manual model remains available, and heavy built-ins load only if explicitly requested. That lets our policies register through `alpasim.models` without carrying the entire built-in driver dependency stack.

The third issue was runtime robustness. In a service-oriented simulator, messages can arrive after a session has closed or after another service has already advanced. The patched driver treats duplicate closes and late image, egomotion, and route updates as warnings instead of fatal errors. This does not change the driving policy; it makes batch rollouts practical.

Figure 1 shows the concrete driver-contract change. The stock path classifies route updates into a compact command and sends that command to the model. The patched path still preserves the command, but also passes route waypoints, ego dynamics, camera context, and pose history into the external plugin. This is the minimum opening needed for WOD-style policies to reason about route geometry without becoming AlpaSim-internal code.

3 Adapter Architecture

Figure 2 shows the implemented architecture. The bridge has two layers: AlpaSim-side enablement and policy-side adaptation.

3.1 AlpaSim-Side Enablement

The patched AlpaSim checkout contains four relevant changes.

Route waypoint propagation. The driver session now stores the most recent route and exposes it through `route_waypoints_for_prediction`. The batch prediction path copies those waypoints into `PredictionInput`. This preserves the route geometry generated by AlpaSim's runtime and makes it available to every external model plugin.

Dependency-light model registry. The model package keeps `BaseTrajectoryModel`, `PredictionInput`, `ModelPrediction`, `DriveCommand`, and `ManualModel` importable without importing all heavyweight backends. Alpamayo and VAM remain available through lazy access when installed. External plugins therefore need only the base driver abstractions.

Asynchronous session hardening. The driver now handles unknown-session close, image, egomotion, and route messages as late or duplicate messages rather than immediate crashes. This matters for long batch runs where the simulator, driver, controller, and renderer do not always stop at exactly the same microsecond.

Local external-driver deployment. The override layer adds deployment and Docker changes for local external-driver use, including host/wizard command generation, ARM-aware deploy selection, minimal Docker dependency groups, and GPU reservation behavior that matches local topologies. The point is not to claim a new simulator backend; it is to make the external-driver path runnable without hand-editing AlpaSim configuration for every experiment.

3.2 Policy-Side Adaptation

The project registers three AlpaSim model entry points: `spotlight_reflex`, `token_dagger_bc`, and `direct_actor_planner`. Each implements AlpaSim's `BaseTrajectoryModel.from_config` factory and returns `ModelPrediction` objects with trajectory points, headings, and structured reasoning metadata.

Minimal-shot reflex plugin. The reflex adapter maps AlpaSim's `DriveCommand` to the policy's route command, extracts route and scene signal, evaluates maneuver candidates, applies route-stability constraints, resamples the selected trajectory to the configured output frequency, and returns headings.

Token BC/DAgger plugin. The learned adapter loads token-policy checkpoints, normalizes geometric features, supports multiple selection modes, and can combine learned logits with route, lane, static-clearance, and actor-clearance constraints. It also supports clamped lateral geometry, hybrid veto modes, actor-axis modes, and per-run selection logs.

Direct actor-planner plugin. The selector-free planner bypasses token logits and samples a small continuous trajectory family. It scores candidates with actor, route, lane, smoothness, progress, speed, and rear-flow terms, then returns the best trajectory under the same AlpaSim driver contract. This makes the harness useful for comparing tokenized and direct planning interfaces inside one simulator.

4 Audited Artifact Surface

The implementation is deliberately split into patched upstream AlpaSim surface area and project-owned adapter surface area. Table 2 lists the core surfaces because this is the main evidence that the contribution is more than a wrapper.

5 Artifact Contents

The artifact is organized as a reproducible integration bundle rather than a single script. Table 3 lists the bundle components and the claim each component supports. This is the checklist a reviewer should use to distinguish the adapter from ordinary experiment glue.

Surface	Implemented role
`alpasim_driver.models.base`	Extends `PredictionInput` with route waypoints so plugins receive route geometry.
`alpasim_driver.main`	Stores submitted routes, forwards route waypoints into batch prediction, and treats late session messages idempotently.
`alpasim_driver.models`	Uses lazy built-in model imports so external plugins can load in dependency-light driver environments.
Wizard and Docker overrides	Adapt local external-driver deployment, host networking, GPU reservations, and minimal runtime dependencies.
`alpasim_signal.py`	Reconstructs route, dynamics, visibility, static hazards, and moving actors from AlpaSim prediction inputs.
`alpasim_spotlight.py`	Wraps the minimal-shot reflex policy as a native AlpaSim trajectory model.
`alpasim_token_bc.py`	Runs learned token BC/DAgger checkpoints with route, lane, actor, and hybrid-veto selection modes.
Direct planner adapter	Runs a selector-free actor-aware trajectory sampler under the same driver contract.
Setup and launch commands	Apply overrides, bootstrap the AlpaSim environment, verify `alpasim.models` discovery, check runtime readiness, and emit reproducible driver/wizard commands.

Table 2: Audited implementation surfaces. The bridge required both patched AlpaSim code and project-owned model adapters.

Bundle component	Reviewer-visible purpose
AlpaSim overrides	Tracks the replacement files under `third_party/alpasim_overrides`: the route-waypoint propagation patch, lazy model imports, and deployment overrides.
Driver configs	Defines native AlpaSim driver YAMLs for the reflex, token BC/DAgger, and direct actor-planner plugins.
Setup command	Applies overrides, bootstraps the AlpaSim environment, installs editable packages, and checks model entry-point discovery.
Readiness command	Checks the non-code runtime assumptions: checkout shape, Docker access, GPU runtime, base image, platform constraints, and scene artifacts.
External-run launcher	Materializes matched driver/wizard commands and launch metadata before optional closed-loop execution.
AlpaSim tests	Verifies plugin registration, signal reconstruction, adapter outputs, checkpoint compatibility, override application, setup planning, and launch command generation.

Table 3: Artifact bundle contents. Each component supports a concrete integration claim that can be inspected independently of the final closed-loop rollout.

6 Signal Reconstruction

The adapter reconstructs a compact policy scenario from `PredictionInput`. It extracts route waypoints from the patched AlpaSim field when present, falling back to a command-proxy route only when route geometry is unavailable. It also extracts speed, acceleration, pose-history length, camera count, a simple visibility-risk signal from camera brightness, and optional structured hazards from fields such as `structured_hazards`, `hazards`, `traffic_hazards`, or `alpasignal`.

Structured hazards are converted into the internal policy representation. Static hazards become obstacles with shape and heading metadata. Moving hazards become actors with velocity, size, heading, and inferred or explicit behavior. Route waypoints become the local lane centerline used by the policy and planner. If no explicit hazard is available but visibility or dynamics indicate a high-risk state, the adapter can add a caution-zone obstacle. This layer is intentionally independent of WOD protobufs and AlpaSim internals: it is the stable policy-facing signal produced by the bridge.

Invariant	Artifact check
Local policies are discoverable by AlpaSim	The package registers `spotlight_reflex`, `token_dagger_bc`, and `direct_actor_planner` under `alpasim.models`; setup verifies registry discovery in the AlpaSim virtual environment.
Route geometry reaches policy code	Signal tests construct prediction inputs with route waypoints and verify that the resulting policy scenario uses `alpasim_waypoints` as its route source.
Scene signal is policy-facing, not protobuf-facing	Tests pass structured hazards through multiple input aliases and verify static obstacles, moving actors, shape metadata, headings, velocities, and inferred behaviors.
Adapters satisfy the trajectory-model contract	The reflex, token BC/DAgger, and direct planner adapters return trajectory arrays, headings, and structured reasoning payloads through the AlpaSim model API.
Learned-policy checkpoints are contract-checked	Token-policy loading verifies token names, feature normalizers, device selection, and selection-log outputs; bad token schemas are rejected.
Setup and launch are reproducible	Setup-script tests cover checkout validation, override application, protobuf compilation, editable install planning, driver command generation, wizard command generation, scene preset loading, Docker/image preflights, and platform guards.

Table 4: Artifact validation targets integration invariants: plugin discovery, route propagation, signal reconstruction, model contract compliance, checkpoint compatibility, and reproducible launch state.

7 Reproducible Execution Path

The integration includes scripts that make the bridge repeatable rather than anecdotal. The setup command validates that `ALPASIM_ROOT` is a real checkout, applies repo-tracked AlpaSim patches and overrides, bootstraps the AlpaSim virtual environment, compiles protobuf modules, installs the required AlpaSim packages editably, installs this repository as a no-dependency editable package, and verifies that the required `alpasim.models` entry points are discoverable.

The readiness command checks the AlpaSim checkout shape, Docker access, platform compatibility, the local AlpaSim base image, NVIDIA container runtime visibility, and scene artifacts. The run launcher then materializes an `external-driver-config.yaml`, `driver-command.sh`, `wizard-command.sh`, and `launch-metadata.json`. It supports named scene presets and policy presets, including learned checkpoint variants and actor-aware planner variants. The driver and wizard are launched as a matched pair: the driver runs from the AlpaSim virtual environment, while the wizard receives the external driver address, topology, scene IDs, log directory, base port, and timeout.

This matters because AlpaSim experiments otherwise depend on a fragile mixture of Hydra overrides, Docker image state, plugin installation state, gated scene artifacts, and host-specific GPU behavior. WOD2ALPAsim makes those assumptions explicit.

8 Artifact Validation

The artifact is organized around invariants that can be checked without reading the entire simulator. Table 4 lists the reviewer-facing checks. They are not performance claims; they are integration claims that establish the bridge is usable and inspectable.

The minimal reviewer path is therefore concrete. First, run the setup command in check-only mode to verify that the AlpaSim checkout can see the local model entry points. Second, run the readiness command to check Docker, GPU runtime, scene artifacts, and platform assumptions. Third, launch in print mode to materialize the exact driver and wizard commands without starting a rollout. Finally, run the AlpaSim integration and setup-script tests to verify the adapter invariants above. This gives

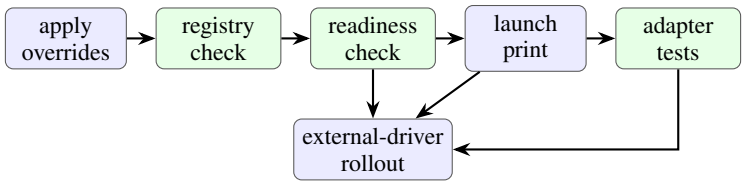

Figure 3: Reviewer-facing artifact path. Setup, readiness, command materialization, and tests isolate integration failures before expensive closed-loop rollouts.

reviewers a bounded artifact audit: if any of these checks fails, the bridge contract is broken in a specific place.

Figure 3 summarizes this audit path. The point is that the artifact is not validated by a single successful rollout. It is validated by a sequence of checks that isolate whether failures come from patch application, plugin discovery, runtime prerequisites, launch materialization, or adapter semantics.

9 What the Adapter Enables

The artifact enables three concrete workflows and one broader engineering precedent.

Closed-loop execution of WOD-style policies. Policies developed around WOD-style route, timing, and trajectory conventions can run as AlpaSim external drivers. The simulator repeatedly queries the plugin and executes the returned trajectory through its controller path.

Shared runtime for heterogeneous policies. The reflex policy, learned token policy, and direct actor planner share one driver contract, one route signal, one trajectory resampling path, and one launch system. This lets policy-interface questions be tested inside a single closed-loop runtime rather than through separate bespoke scripts.

Inspectable transfer infrastructure. Because the bridge separates patched upstream work from project-owned adapters, it is clear which claims depend on AlpaSim modifications and which depend on policy logic. That separation is useful for workshop review: the contribution can be audited as an artifact, not merely described as a conceptual adapter.

A precedent for future AlpaSim adapters. The same pattern can support future bridges for other dataset or benchmark contracts: preserve simulator-native route geometry at the driver boundary, avoid dataset protobufs inside policy code, expose only compact policy-facing signal, keep policies registered as ordinary AlpaSim model plugins, and make setup plus launch assumptions executable. In this sense, WOD2ALPAsim is both a WOD bridge and a reference pattern for building additional AlpaSim adapters.

10 Limitations

WOD2ALPAsim does not claim to reconstruct the full WOD world state inside AlpaSim. Its primary claim is execution compatibility: making WOD-style and project policies run as native AlpaSim drivers. Richer traffic reconstruction, broader WOD subset release, dataset-terms packaging, and standard public baselines are natural next steps. The current artifact already exposes the important systems boundary: route geometry, trajectory timing, plugin loading, deployment, and simulator-driver interaction.

11 Conclusion

WOD2ALPAsim is a full-stack adapter, not a thin wrapper. The work patched AlpaSim's driver input contract, deployment path, and import behavior; registered project policies as AlpaSim model plugins; reconstructed route and scene signal for WOD-style policies; and added reproducible setup, readiness, and launch tooling. The result is a practical closed-loop harness for running minimal-shot, learned token, and actor-aware planning policies under AlpaSim's simulator runtime. That is the workshop contribution: the engineering bridge pattern that turns dataset-style trajectory policies into executable closed-loop drivers. Future AlpaSim adapters should be built at stable driver, signal, plugin, and launch seams rather than as one-off simulator forks or dataset-specific policy rewrites.

References

[1] P. Sun, H. Kretzschmar, X. Dotiwalla, A. Chouard, V. Patnaik, P. Tsui, J. Guo, Y. Zhou, Y. Chai, B. Caine, et al. Scalability in perception for autonomous driving: Waymo open dataset. In *IEEE Conference on Computer Vision and Pattern Recognition*, pages 2446–2454, 2020.

[2] NVIDIA Research. AlpaSim: Autonomous driving simulation for policy evaluation, 2026. Software release v2026.4.